



CMPT 295

Unit - Machine-Level Programming

Lecture 15 – Assembly language – Program Control – **cmovX**
Iterative Statements – Loops

Last Lecture

- In **C**, we can control the execution flow of a program
 1. Conditionally
 - Conditional statements: **if/else**, **switch**, ...
 - Iterative statements: **for** loops, **while** loops, ...
 2. Unconditionally
 - Functions calls
- In **x86-64** assembly, we can also control the execution flow of a program
 - **cmp*** instruction (*compare*)
 - **jX** instructions (*jump*)
 - **call** and **ret** instructions

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + **`cmovX`**
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

Example: alternate `int max(int x, int y)`

in C:

```
int max(int x, int y) {  
    int result = x;  
    if (y > x)  
        result = y;  
    return result;  
}
```

version 2 – with `cmovx` instruction - mine

in assembly:

max:

```
# x in %edi, y in %esi, result in %eax  
movl %edi, %eax # result = x  
cmpl %edi, %esi # if y > x then  
cmovg %esi, %eax # return result = y  
ret
```

version 1 – with `jlx` instruction

in assembly:

max:

```
# x in %edi, y in %esi, result in %eax  
movl %edi, %eax # result = x  
cmpl %edi, %esi # if y <= x then  
jle endif # return x (result)  
movl %esi, %eax # otherwise return y
```

endif:

ret

Example: alternate `int max(int x, int y)`

in C:

```
int max(int x, int y) {  
    int result = x;  
    if (y > x)  
        result = y;  
    return result;  
}
```

version 3 - with `cmovx` instruction – from gcc (-O1)

in assembly:

`max:`

```
# x in %edi, y in %esi, result in %eax  
cmpl    %esi, %edi # if x >= y then ...  
movl    %esi, %eax # result = y  
cmovge  %edi, %eax # ... return result = x  
ret
```

Conditional move instruction **cmovX**

Here is what **C** code looks like when using conditional operator:

```
result = Boolean conditiontest ? val2 : val1;  
return result;
```

Here is what the logic of assembly code looks like when using **cmovX**:

```
result = val1;  
if (test) result = val2;  
return result;
```

Condition

X
e
ne
s
ns
g
ge
l
le
a
b

➤ Example: **cmovl Src, Dest**

alternative: `int abs(int x)`

in **C**:

```
int abs(int x){  
    if ( x < 0 )  
        x = -x;  
    return x;  
}
```

in assembly:

abs:

```
# x in %edi, result in %eax  
movl %edi, %eax # result = x  
negl %edi       # x = -x  
testl %eax, %eax # if x < 0 then  
cmovs %edi, %eax # result = -x  
ret
```

Advantage of conditional move **cmovX**

Note about **branching**:

- Branches are very disruptive to instruction flow through microprocessor pipelines (we shall see this in our next unit)
- However, since conditional moves (**cmovX**) do not require control transfer (no branching/jumping required), they are less disruptive
- So, **gcc** uses **cmovX** often, but only when it is safe

What do we mean by “safe”?

➤ In `result = test ? aVal : anotherVal;` **both values** (`aVal` and `anotherVal`) **are computed** so their computation must be **safe**

➤ Example of **unsafe** computations:

1. Expensive computations `val = Test(x) ? Hard1(x) : Hard2(x);`

➤ Only makes sense when computations are very simple

2. Risky computations `val = ptr ? *ptr : 0;`

➤ Only makes sense when computations do not crash the application

3. Computations with side effects `val = x > 0 ? x*=7 : x+=3;`

➤ Only makes sense when computations do not have side effects

While loop – “*coding the false condition first*”

`int x` and `int y` are arguments to function

in C:

```
while (x < y) {  
    // stmts  
}  
return;
```

in assembly:

loop:

endloop:

ret

Loop Pattern 1

```
loop:  
    if cond false  
        goto done:  
    stmts  
    goto loop:  
done:
```

While loop – “jump-to-middle”

`int x` and `int y` are arguments to function

in C:

```
while (x < y) {  
    // stmts  
}  
return;
```

in assembly:

```
loop:  
    # stmts  
test:  
  
ret
```

Loop Pattern 2

```
goto test:  
loop:  
    stmts  
test:  
    if cond true  
        goto loop:  
done:
```

Do While loop – “jump-to-middle”

`int x` and `int y` are arguments to function

```
in C:  
do {  
    stmts  
} while (x < y);  
return;
```

in assembly:

```
loop:  
    # stmts  
test:  
  
ret
```

Loop Pattern 2

```
goto test:  
loop:  
    stmts  
test:  
    if cond true  
        goto loop:  
done:
```

For loop

In C:

```
for (i = 0; i < n; i++){  
    // stmts  
}  
return;
```



```
while (i < n) {  
    // stmts  
}  
return;
```



In Assembly:

```
xorl %ecx, %ecx
```

```
loop:
```

```
endloop:
```

```
ret
```

Compiler can produce different combinations of instructions when assembling the same **C** code.

Summary

- In **x86-64 assembly**, there are *no conditional statements*, however, we can alter the execution flow of a program by using ...
 - **cmp*** instruction (compare)
 - **jX** instructions (jump)
 - **cmovX** instructions (conditional move)
- In **x86-64 assembly**, there are *no iterative statements*, however, we can alter the execution flow of a program by using ...
 - **cmp*** instruction
 - **jX** instructions (jump)
- Microprocessor uses the **condition codes** to decide whether a ...
 - **jX** instruction (conditional jump) is to be executed or a
 - **cmovX** instruction (conditional move) is to be executed

Instructions such as arithmetic and logical instructions, **cmp*** and **test*** set condition codes

Next Lecture

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, move operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations